Digital Signal Processing

A Complete, From-Zero Study Guide

Understand the concept first — memorize the MATLAB command second.

MD. SAJEEB KHAN

This guide walks through every topic in your DSP lab — signals, systems, convolution, correlation, the Z-transform, Fourier analysis, the DFT/FFT, and filtering — building each idea from intuition, to formula, to worked example, to the MATLAB code your book uses.

Table of Contents

0. What Is DSP? The Big Picture

DSP stands for Digital Signal Processing. A signal is simply information represented as something that changes — sound, temperature, an ECG trace, voltage, or an image all count. In this lab you almost always work with a discrete-time signal, written x[n], where n is the sample number (not a moment in continuous time).

```
n :   0   1   2   3   4
x[n]: 2   4   1   3   5

x[0] = 2,  x[1] = 4,  x[2] = 1, ...

MATLAB:
n = 0:4;
x = [2 4 1 3 5];
stem(n,x)     % stem() is used because a discrete signal is a set of
              % individual samples, not a continuous curve.
```

The whole subject as one story

Every experiment in your lab is a variation on the same ten-step story. Keep this in your head as the map; every topic below fills in one box of it.

```
Step 1  Make a signal              impulse, step, ramp, sinusoid, ...
Step 2  Manipulate it               add, multiply, reverse, even/odd
Step 3  Put it through a system     x[n] -> SYSTEM -> y[n]
                                     (linear? time-invariant? stable?)
Step 4  Find the system output      y[n] = x[n] * h[n]   (convolution)
Step 5  Compare signals             correlation ("how similar?")
Step 6  Change domain               Z-transform  or  Fourier/DTFT
Step 7  Look at frequency           Fourier, DTFT, DFT, FFT
Step 8  Understand periodic signals Fourier Series, harmonics, Gibbs
Step 9  Make computation efficient  FFT
Step 10 Design filters              Butterworth, poles/zeros
```

> **Golden rule**
>
> For every MATLAB function below, learn it in this order: what the concept means -> what your book's code does -> why the code works -> what to write in the lab. Don't jump straight to the command.

1. Basic Discrete-Time Signals

Think of these as the alphabet of DSP — every experiment starts from one of these building blocks.

1.1 Unit Impulse δ[n]

Only one sample, at n = 0, is nonzero. Think of it as a single clap: silence, then one instant, then silence again.

```
n:    ... -2 -1  0  1  2 ...
δ[n]:  ...  0  0  1  0  0 ...

MATLAB:
n = -5:5;
x = double(n == 0);
stem(n,x)
```

1.2 Unit Step u[n]

The step turns on at n = 0 and stays on — like flipping a light switch.

```
n:    -3 -2 -1  0  1  2  3
u[n]:  0  0  0  1  1  1  1

MATLAB:
x = double(n >= 0);
```

Impulse vs Step

Impulse = ONE sample turns on then off again. Step = turns on and STAYS on. This is the #1 mix-up on this topic.

1.3 Ramp r[n] = n

Increases by the same amount every step — like walking upstairs one step at a time.

```
n:    0 1 2 3 4
r[n]: 0 1 2 3 4

MATLAB:
x = n;
```

1.4 Parabolic p[n] = n²/2

Here the increase itself keeps getting bigger, unlike the ramp where it stays constant.

```
n:      0   1   2   3   4
n²/2:   0  .5   2  4.5   8

MATLAB:
x = 0.5*n.^2;
```

Ramp vs Parabolic

Ramp grows proportional to n. Parabolic grows proportional to n² — it accelerates.

1.5 Exponential $x[n] = a^n$

Everything about this signal's shape depends on the value of a:

Range of a	Behaviour
$0 < a < 1$ (e.g. 0.5)	Decays smoothly toward 0
$a > 1$ (e.g. 2)	Grows without bound
$-1 < a < 0$ (e.g. -0.5)	Alternates sign while shrinking
$a < -1$	Alternates sign while growing

Rule of thumb: whenever you see an exponential question, look at a first.

1.6 Sinusoidal Signal $x[n] = A\cos(2\pi fn)$

Two parameters matter: A (amplitude) controls the height of the wave; f (frequency) controls how many oscillations happen per sample.

```
A = 2; f = 0.2;
x = A*cos(2*pi*f*n);

Increase A -> taller wave
Increase f -> faster wave
```

1.7 Swept-Frequency Signal $x[n] = \cos(a \cdot n^2)$

Unlike an ordinary cosine, the frequency here is not constant — it starts slow and speeds up. This is different from $\cos(2\pi fn)$, where f never changes.

```
Ordinary cosine:  ~ ~ ~ ~ ~ ~ ~ ~   (constant spacing)
Swept frequency:  ~  ~  ~~ ~~ ~~~~ ~~~~~~~~  (spacing shrinks)
```

2. Signal Operations

Now that we have signals, what can we do to them?

2.1 Addition and Multiplication

For x1 = [1 2 3] and x2 = [4 5 6]:

Operation	Result	MATLAB
Addition x1 + x2	[5 7 9]	x1 + x2
Multiplication x1·x2 (element-wise)	[4 10 18]	x1 .* x2

Why .* and not *?

In MATLAB, * means matrix multiplication. .* means "multiply corresponding elements." For signals you almost always want .*

2.2 Time Reversal

Time reversal flips the signal around n = 0:

```
Original x[n]:  1 2 3 4
Reversed x[-n]: 4 3 2 1

MATLAB:
xr = fliplr(x);
```

Lab detail

fliplr() only flips the MATLAB array — it does not automatically relabel the sample indices for negative n. When an experiment needs the true x[-n] indexing (e.g. the even/odd decomposition or DTFT time-reversal experiments), you must also track the index shift, not just the array order.

2.3 Even and Odd Decomposition

Every signal can be split into an even part and an odd part:

```
x[n] = xe[n] + xo[n]

xe[n] = ( x[n] + x[-n] ) / 2      <- EVEN uses PLUS
xo[n] = ( x[n] - x[-n] ) / 2      <- ODD uses MINUS
```

Type	Test	Picture
Even	x[-n] = x[n]	Mirror image left/right of n = 0

Odd	x[-n] = -x[n]	Opposite sign on the other side of n = 0

Memory trick: EVEN → PLUS, ODD → MINUS. Note some experiments (e.g. the DFT-based even-component test) use a periodic/circular version of x[-n] rather than a plain array flip — the exact implementation depends on context.

3. Systems and Their Properties

A system takes an input signal and produces an output signal — like a washing machine turning dirty clothes into clean clothes.

```
x[n]  ->  [ SYSTEM ]  ->  y[n]

MATLAB representation (numerator/denominator coefficients):
y = filter(num, den, x);
```

3.1 Impulse Response h[n]

Feed the system an impulse and see what comes out — that output is called the impulse response, h[n]. For an LTI (linear, time-invariant) system, knowing h[n] tells you how the system responds to any input, via convolution:

```
δ[n]  ->  SYSTEM  ->  h[n]

y[n] = x[n] * h[n]

MATLAB:
h = impz(num, den, N);
```

3.2 Linearity

A system T is linear if combining inputs first and processing gives the same result as processing first and then combining:

```
T{ a*x1 + b*x2 } = a*T{x1} + b*T{x2}
```

In plain English: combine first, then process = process first, then combine. Your book tests this by computing both sides and checking that their difference is approximately zero.

3.3 Time Invariance

A system is time-invariant if delaying the input simply delays the output by the same amount, with no other change:

```
x[n]     -> SYSTEM -> y[n]
x[n - D] -> SYSTEM -> y[n - D]      (if time-invariant)
```

Your book tests this by creating a delayed input (zero-padding) and comparing the resulting output to the delayed original output.

3.4 Stability

A system is stable if a reasonable (bounded) input never produces an uncontrollably huge output. For an LTI system the standard test is:

```
sum( |h[n]| )  <  infinity      over all n
```

Don't mix these up

LINEARITY -> combination (addition/scaling). TIME INVARIANCE -> shifting in time. STABILITY -> does the output blow up? Three different questions, three different tests.

4. Convolution

Probably the single most important concept in the whole lab. If you know the input x[n] and the system's impulse response h[n], the output is:

```
y[n] = x[n] * h[n]        (* here means convolution, NOT ordinary multiplication)
```

4.1 Why does convolution exist?

Think of dropping a stone into water: one disturbance spreads outward as h[n]. If you drop many stones according to x[n], the total water response is the sum of many shifted, scaled copies of h[n]. That sum is convolution.

4.2 Worked example — Linear Convolution

Slide one sequence across the other; at each position multiply overlapping values and add them.

```
x = [1 2]
h = [2 1]

Position 1:  1*2               = 2
Position 2:  1*1 + 2*2 = 1 + 4 = 5
Position 3:  2*1               = 2

y = [2 5 2]

MATLAB:  y = conv(x,h);
Output length = Lx + Lh - 1 = 2 + 2 - 1 = 3
```

4.3 Circular Convolution

Looks similar, but the sequences wrap around like points on a circle instead of ending.

	Linear convolution	Circular convolution
Wrapping	None	Yes — samples wrap around
Output length	Lx + Lh - 1	Fixed length N
MATLAB	conv()	cconv(x,h,N)
Nature	Normal convolution	Periodic convolution

4.4 Convolution via the DFT (and why zero-padding matters)

A core DSP identity: convolution in time corresponds to multiplication in frequency.

```
x[n] --FFT--> X[k]
h[n] --FFT--> H[k]
                X[k] * H[k]
                     |
                   IFFT
                     |
```

```
                      y[n]
```

But multiplying two DFTs and taking the IDFT naturally gives circular convolution, not linear convolution. To get a true linear convolution result out of the DFT, you must zero-pad both sequences to length N = Lx + Lh - 1 before the FFT:

```
x -> zero-pad to N   \
h -> zero-pad to N    >  FFT -> multiply -> IFFT -> linear convolution
                     /

Without correct zero-padding: DFT multiplication -> circular convolution
With N = Lx + Lh - 1:          DFT multiplication -> linear convolution
```

5. Correlation

Correlation looks like convolution but answers a different question.

Convolution asks	Correlation asks
“What output does this combination/system produce?”	“How similar are these signals when I shift one relative to the other?”

Type	Compares	MATLAB
Autocorrelation	A signal with itself	xcorr(x)
Cross-correlation	Two different signals	xcorr(x,y)

Memory trick: AUTO = same signal, CROSS = two different signals.

6. The Z-Transform

The Z-transform re-represents a discrete-time signal in another mathematical language, X(z), similar to translating a sentence into another language: same information, different form. Some system problems become much easier to solve in the Z-domain.

6.1 Forward and Inverse

Direction	Meaning	MATLAB
Forward	x[n] → X(z)	X = ztrans(x);
Inverse	X(z) → x[n]	x = iztrans(X);

6.2 Residues / Partial Fractions

For a complicated rational expression N(z)/D(z), partial fractions break it into simpler pieces such as A/(z-p1) + B/(z-p2).

```
[r,p,k] = residue(num,den);

r -> residues
p -> poles
k -> direct term
```

Don't confuse

residue() is not the inverse Z-transform itself — it's a method that helps you decompose the expression into pieces you can then invert term by term.

6.3 Long Division

Used to get the first few terms of an inverse Z-transform, by dividing polynomials the same way you divide ordinary numbers in long division.

```
[h,rem] = deconv(num_padded,den);
```

7. Fourier Transform and DTFT

A different question from convolution/correlation: what frequencies are inside this signal?

```
TIME DOMAIN  x(t)
     | Fourier Transform
FREQUENCY DOMAIN  X(f)
```

7.1 Fourier Transform vs DTFT

	Fourier Transform	DTFT
Used for	Continuous-time signals x(t)	Discrete-time signals x[n]
Result	X(ω)	X(e^{jω})
MATLAB	fourier(x)	freqz()

7.2 Magnitude and Phase

DFT/DTFT results are generally complex numbers. For X = 3 + j4:

```
Magnitude: |X| = sqrt(3²+4²) = 5        MATLAB: abs(X)
Phase:     ∠X = atan(4/3)               MATLAB: angle(X)
```

abs() answers “how big?”; angle() answers “which direction/phase?”

8. DTFT Properties

These are not new transforms — they are properties of the DTFT you already know.

8.1 Time-Shifting Property

If x[n] → X(e^{jω}), then x[n - n0] → e^{-jωn0} X(e^{jω}). A shift in time changes only the phase; the magnitude is unaffected.

8.2 Frequency-Shifting Property

Multiplying a signal by a complex exponential shifts its spectrum:

```
x[n]*e^{jω0 n}  ->  X(e^{j(ω - ω0)})

Multiply in time  ->  shift in frequency

MATLAB: exp(1j*w0*n).*x
```

8.3 Time Reversal Property

If x[n] → X(e^{jω}), then x[-n] → X(e^{-jω}): reversing the signal in time reverses (mirrors) its spectrum. Your book combines fliplr() with an index correction, since array-reversal alone does not automatically represent true negative-time indexing.

8.4 Convolution Property

One of the most useful relationships in all of DSP:

```
TIME                        FREQUENCY
convolution        <->      multiplication
multiplication      <->      convolution
```

9. Fourier Series

Do not confuse this with the Fourier Transform. Fourier Series applies to periodic signals and says: a periodic waveform can be built by adding simple sinusoids (harmonics).

```
square wave  ≈  sin(t) + sin(3t)/3 + sin(5t)/5 + sin(7t)/7 + ...
```

9.1 Harmonics

If the fundamental frequency is f: 1st harmonic = f, 2nd = 2f, 3rd = 3f, and so on. A square wave is built mainly from odd harmonics (f, 3f, 5f, 7f, ...). More harmonics added → a progressively better approximation to the square wave.

9.2 Gibbs Phenomenon

Near a sharp jump/discontinuity, the Fourier-series approximation overshoots and oscillates. Adding more harmonics improves the approximation everywhere else, but the overshoot at the edge does not disappear — that persistent ripple is the Gibbs phenomenon.

10. DFT and FFT

Now we need something a computer can actually compute: a finite sequence in, a finite sequence out.

```
x[0], x[1], ..., x[N-1]   --DFT-->   X[0], X[1], ..., X[N-1]
```

10.1 DFT vs DTFT

DTFT	DFT
Frequency variable is continuous	Only a finite set of frequency samples
Theoretical transform	Practical / computational transform
Spectrum is a continuous curve	Spectrum is N discrete points

10.2 DFT vs FFT — the #1 confusion

> **FFT is not a different transform**
>
> DFT is the mathematical operation (WHAT you calculate). FFT is a fast algorithm for calculating the DFT (HOW you calculate it quickly). Think: DFT = the destination, FFT = the fast road to that destination.

If an exam question says “using the DFT matrix,” use dftmtx(); if it says “calculate the DFT,” fft() is the normal, convenient MATLAB approach.

10.3 IDFT / IFFT

```
X = fft(x);          % forward
x_recovered = ifft(X); % inverse
```

10.4 The DFT Matrix

Your book also demonstrates the DFT as an explicit matrix operation, to show the mathematics behind fft():

```
D = dftmtx(N);
X = x * D;          % forward DFT via matrix multiplication
x = X * conj(D)/N;  % recovers the sequence
```

11. Parseval's Theorem

Energy does not disappear just because we changed domains — time-domain energy equals frequency-domain energy (up to numerical error):

```
Time-domain energy:  Et = sum( |x[n]|² )
DFT-domain energy:   Ef = (1/N) * sum( |X[k]|² )

Et  ≈  Ef
```

12. Circular Shift

A normal delay pads with zeros; a circular shift wraps the values around instead:

Type	[1 2 3 4] becomes
Normal delay	[0 1 2 3 4]
Circular shift	[3 4 1 2] (values wrap around)

```
MATLAB:
y = circshift(x,-M);

% then compare fft(x) and fft(y) magnitude/phase
```

13. Filters — The Butterworth Filter

A filter keeps some frequencies and rejects others — like a security gate that only lets certain frequencies through.

```
Frequency:   LOW   MID   HIGH
              ✗     ✓     ✗        <- example: bandpass filter
```

The Butterworth filter is a specific design with a smooth (maximally flat) magnitude response. Your book's workflow:

Function	Purpose
buttord(wp, ws, Rp, Rs)	How many filter stages are needed? Returns the filter order n and cutoff wc.
butter(n, wc)	Actually designs the Butterworth filter, returning zeros/poles/gain.
zplane(z, p)	Plots the zeros and poles on the complex plane.

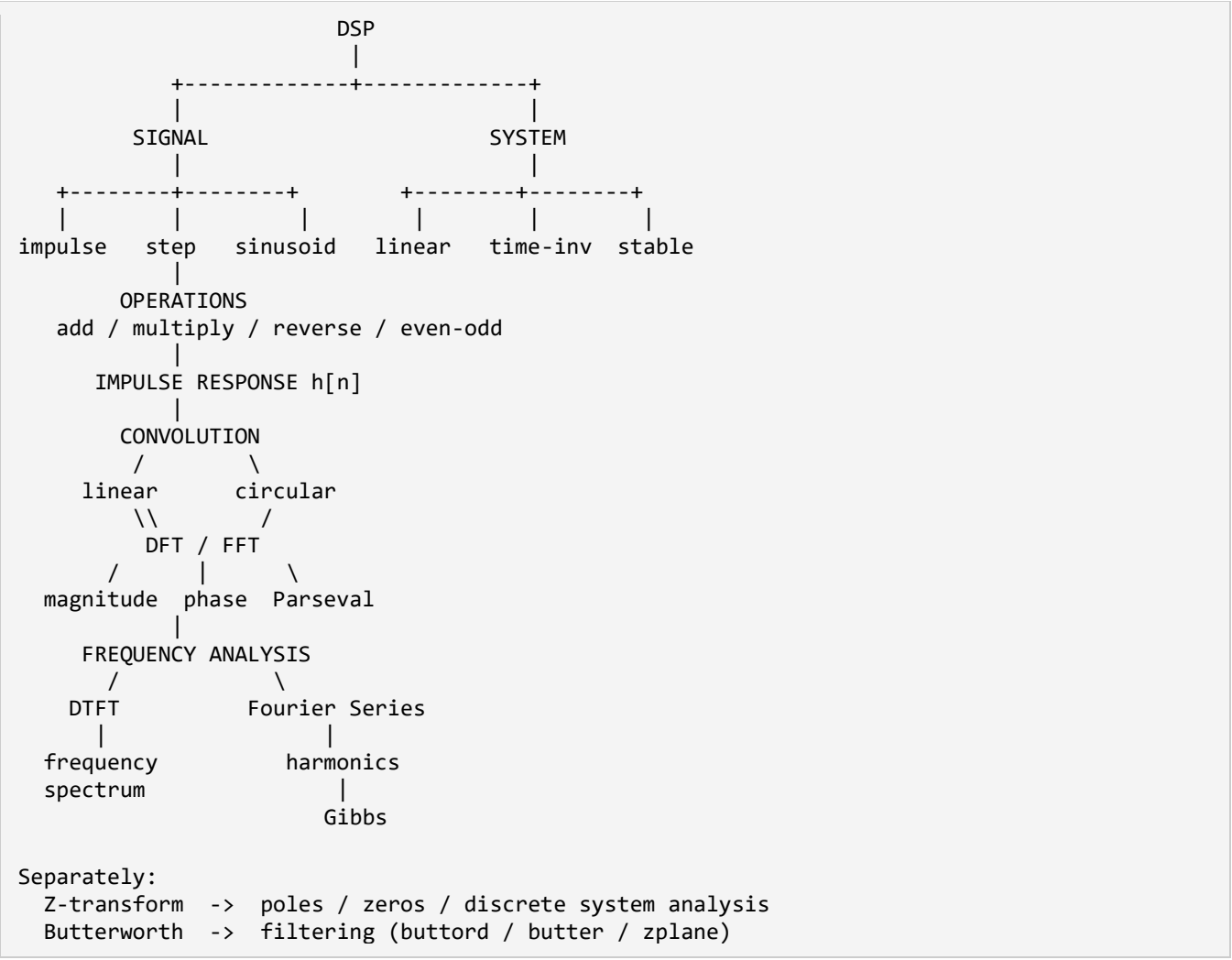
14. The Big “Don't Confuse These” Reference

This is the single most exam-useful table in this guide.

Looks similar	Actual difference
Impulse vs Step	Impulse = one sample; step = stays ON
Ramp vs Parabolic	Ramp $\propto n$; parabolic $\propto n^2$
Addition vs Multiplication	+ vs .*
Even vs Odd	+ vs − with the reversed signal
Linearity vs Time Invariance	combine vs shift
Time Invariance vs Stability	delay behaviour vs boundedness
Convolution vs Multiplication	sliding interaction vs sample-by-sample product
Convolution vs Correlation	system output vs similarity
Linear vs Circular Convolution	no wrapping vs wrapping
Autocorrelation vs Cross-correlation	same signal vs two signals
Z-transform vs Fourier/DTFT	complex system-domain vs frequency analysis
Fourier Transform vs Fourier Series	frequency content vs harmonic build-up of a periodic signal
Fourier Transform vs DTFT	continuous-time vs discrete-time
DTFT vs DFT	continuous frequency vs discrete frequency samples
DFT vs FFT	the transform vs the fast algorithm for it
DFT vs IDFT	forward vs reverse
Time shift vs Time reversal	move vs flip
Time shift vs Frequency shift	move the signal vs move the spectrum
Magnitude vs Phase	size vs angle
conv() vs cconv()	linear vs circular
fft() vs freqz()	DFT/FFT vs frequency response (DTFT-style)
ztrans() vs fourier()	Z-domain vs Fourier-domain

15. One Diagram to Remember Everything

If you only look at one thing before the lab test, make it this map. Once you understand it, MATLAB functions stop looking like random commands — they become tools for doing specific, recognizable jobs.



How to actually study this

For each experiment in your lab manual, work through: concept -> intuition -> formula -> a small worked numerical example -> what the graph should look like -> the MATLAB code -> why that code produces that result -> the common mistake -> how an examiner might phrase the question. That sequence, repeated per topic, is what turns this from a list of commands into real understanding.